

ALGORITMIA BÁSICA
Grado en Ingeniería Informática y Doble Grado en Matemáticas-Ingeniería Informática
Examen extraordinario (25/06/2024)

- Resolver los ejercicios en **hojas separadas**. Todas las hojas deben llevar apellidos, nombre y número de orden.
- Si no da tiempo a resolver alguna parte, se tendrá en cuenta el indicar en *detalle* cómo se resolvería. Hacer un procedimiento o función que resuelve el problema **no** es resolver el ejercicio: se trata de **incidir en los aspectos de especificación y diseño**.
- Si algo no se especifica explícitamente, indicar las decisiones adoptadas (ej.: “como no se indica lo contrario, supongo que ...”).
- **Comentar** los algoritmos adecuadamente.
- No se admitirán soluciones a lápiz ni con bolígrafo rojo.
- Duración máxima del examen: **3 horas**

Ej. 1 — (2,5 puntos) Tenemos una pared de longitud L y suficientes estanterías para colgar de dos tipos de longitud $l_1, l_2 \leq L$ ($l_1 \neq l_2$); las más largas son más baratas. El objetivo es poner estanterías en la pared de forma que se minimice la longitud que queda libre y, en caso de soluciones con la misma longitud libre, preferimos las más baratas.

Ejemplos:

Entrada	Salida	Comentarios
$L = 24, l_1 = 3, l_2 = 5$	3 3 0	Tres unidades de cada tipo y quedará 0 espacio libre. $3 \cdot 3 + 3 \cdot 5 = 24$. Otra solución podría haber sido 8 0 0, pero como las estanterías más largas son más baratas la solución 3 3 0 es mejor.
$L = 29, l_1 = 3, l_2 = 9$	0 3 2	0 unidades de l_1 , 3 unidades de l_2 y quedan 2 unidades de medida sin estantería. $0 \cdot 3 + 3 \cdot 9 + 2 = 29$
$L = 24, l_1 = 4, l_2 = 7$	6 0 0	Con 6 unidades de l_1 dejamos la pared cubierta. $6 \cdot 4 + 0 \cdot 7 + 0 = 24$

Se pide:

- a) Diseñar una solución eficiente mediante el esquema voraz.
- b) Hacer una estimación del coste en tiempo de la solución.

Ej. 2 — (2,5 puntos) Dada una vara que mide n centímetros y un vector que indica los precios de todas las varas de tamaño menor o igual que n , determinar cuál es el máximo valor que podemos conseguir cortando la vara y vendiendo los trozos obtenidos.

Ejemplos:

- Si la longitud es 8 y los precios son:

longitud	1	2	3	4	5	6	7	8
precio	1	5	8	9	10	17	17	20

El máximo valor que podemos obtener es de 22 (cortando la vara en dos piezas de longitud 2 y 6).

- Si la longitud es 8 y los precios son:

longitud	1	2	3	4	5	6	7	8
precio	3	5	8	9	10	17	17	20

El máximo valor que podemos obtener es de 24 (cortando la vara en 8 piezas de longitud 1).

Se pide:

- Si $\text{corteVara}(n)$ es el mejor beneficio que podemos obtener con una vara de longitud n , escribir la ecuación en recurrencias que permite calcular este valor.
- Diseñar un algoritmo que resuelva el problema del cálculo de dicho valor utilizando las ecuaciones recursivas.
- Analizar la eficiencia de la solución propuesta. No hace falta un análisis formal, basta con una idea genérica del coste en tiempo y espacio en función de n .
- Proponer una solución eficiente en tiempo para resolver el problema.

Ej. 3 — (2,5 puntos) En el *Cinelandia*, la última fila tiene asientos numerados del 1 al N . El asiento 1 está pegado a la pared, por lo que la única entrada a la fila es por el asiento N . Cuando alguien quiere llegar a algún asiento k , debe pasar por los asientos $N, N-1, \dots, k+1$, y cualquiera que ya esté sentado en esos asientos debe ponerse de pie para dejar pasar a la nueva persona (y luego se vuelven a sentar). Para la próxima proyección, tienes una lista $l[1..n]$ del orden en que llegarán los $n \leq N$ espectadores que tendrán que sentarse en la última fila, con sus números de asiento. Por ejemplo, la lista $[3, 1, 2, 5, 4, 9, 10]$ (7 espectadores) indica que la primera persona en llegar tendrá que sentarse en el asiento 3, la segunda en el asiento 1, etc. La primera persona en llegar tendrá que ponerse en pie dos veces (para dejar pasar a la segunda — con asiento 1 — y a la tercera — con asiento 2) y la cuarta persona en llegar (con asiento 5) tendrá que ponerse en pie una vez (para dejar pasar a la quinta — con asiento 4).

- Dada la lista $l[1..n]$, escribe un algoritmo (en pséudo-código) *eficiente** para calcular el número total de casos de ponerse de pie: la misma persona puede levantarse varias veces y todas esas cuentan por separado.
- Calcula el coste asintótico $O(\cdot)$ en tiempo del algoritmo considerando la longitud n de la lista en entrada.

*Observación: En este problema algoritmo *eficiente* quiere decir con un coste asintótico mejor que $O(n^2)$, donde n es la longitud de la lista.

Ej. 4 — (2,5 puntos) Hay que distribuir n miembros de un comité alrededor de una mesa de trabajo redonda con n sillas. Se dispone de información sobre la calidad de la relación entre los miembros: sea C la matriz (simétrica) de conflictos, $C[x, y]$ es un valor comprendido entre 0 (sintonía perfecta) y 10 (aversión profunda) que indica el nivel de conflicto entre x y y . Se quiere aplicar el esquema de ramificación y poda de mínimo coste para determinar una colocación de los miembros en la mesa que minimice el nivel de conflicto general. El conflicto general se calcula sumando los niveles de conflicto de los miembros sentados en posiciones adyacentes. Se pide definir:

- La representación de las soluciones y el árbol de búsqueda;

- b)** La función de coste $c(x)$;
- c)** La función de estimación del coste $\hat{c}(x)$;
- d)** La función de poda $\mathcal{U}(x)$.